



3

Управление изменениями программных продуктов





Изменение

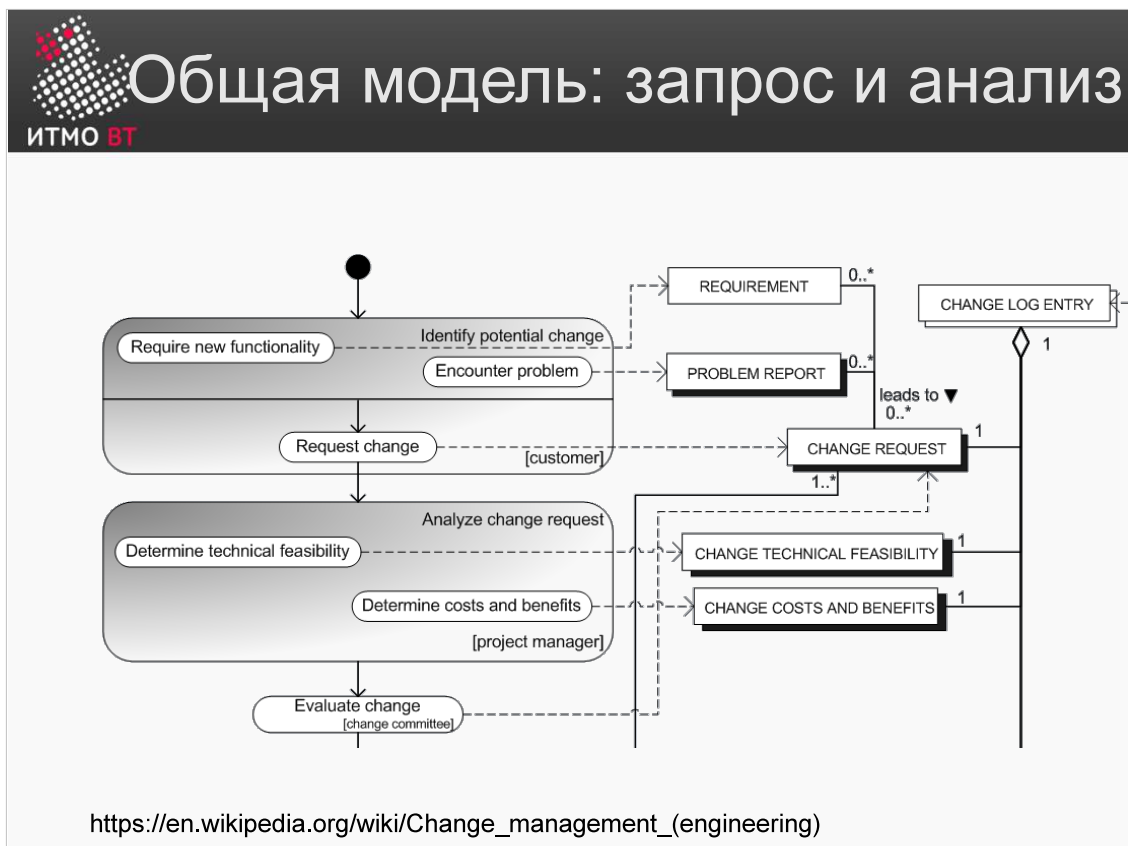
- Контролируемое, журналируемое обновление системы.
- Атрибуты изменения:
 - Идентификатор, дата, ответственный, описание, журнал изменения.
- Управлению изменениями подлежат:
 - Требование новой функциональности.
 - Нефункциональные требования.
 - Исправление дефектов.
 - Другие артефакты архитектуры, анализа, дизайна.

При увеличении количества людей, работающих над проектами, накапливающиеся изменения нарастают как снежный ком, и контроль и управление ими приходится организационно контролировать. Этим занимается дисциплина, называемая управлением изменениями (change management). В экономике управление изменениями (например, в организации, ведущей хозяйственную деятельность), является основополагающим инструментом адаптации к меняющимся условиям бизнеса.

У сложных систем (в том числе и программных) невозможно, или очень сложно, менять систему одновременно в нескольких компонентах. Поэтому вносимые изменения обычно выстраиваются в последовательность отдельных действий, которые необходимо контролировать и фиксировать атрибуты каждого такого изменения в журнале.

Системы контроля версий являются примерами систем учета изменений и позволяют вести такие журналы. При помощи журнала можно просмотреть историю правок и оценить их влияние на поведение системы. Каждое изменение имеет атрибуты: идентификатор, дату, ответственного, описание, связанный с ним журнал изменения и т.д.

В разработке ПО не только изменения программного кода могут попадать под контроль — отдельному управлению изменениями обычно подлежат сами требования к ПО, выявленные дефекты, артефакты архитектуры, анализа и дизайна.



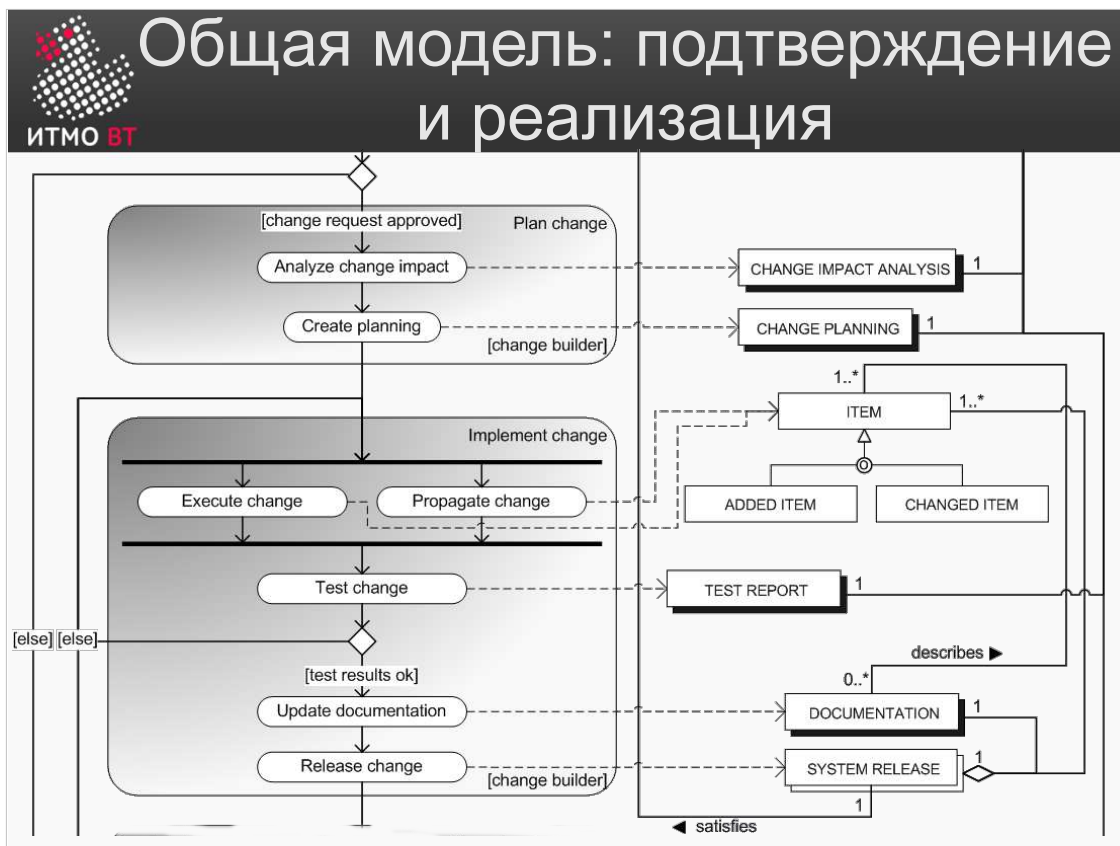
На слайде приведена модель процесса управления изменениями, который осуществляется в области разработки систем и, в том числе, в инженерии программного обеспечения. Все современные средства по управлению изменениями реализуют близкий к приведённому процесс.

В левой части указаны активности (деятельности) ролей процесса с целью формирования и изменения набора артефактов для ведения изменения. Справа показаны сами артефакты изменения, участвующие или возникающие в процессе. Следует заметить, что данная модель представляет собой гибрид двух UML-диаграмм — диаграммы деятельности и диаграммы классов верхнего уровня.

Заказчик (Customer) вносит изменения в систему ради появления новой функциональности (Require new functionality) или исправления выявленных им проблем (Encounter problem). Каждая из этих деятельностей приводит к созданию документа или материальных артефактов: Requirements (требования к системе) Problem report (отчёт о проблеме или ошибке). Оба типа запросов формируют т.н. запрос на изменение (Change Request, часто используется акроним CR), и на каждый такой запрос формируются записи в Change Log Entry.

После этого запросы на изменения поступают к менеджеру проекта (Project manager). У менеджера есть две необходимых деятельности, которые он должен осуществить в процессе анализа запроса на изменения: он определяет техническую необходимость и осуществимость изменений (technical feasibility), а также анализирует стоимость и преимущества для пользователя (costs and benefits). При этом первый фактор формирует документ или артефакт Change Technical Feasibility, а второй — Change Costs and Benefits. Всё это отражается в журнале изменений.

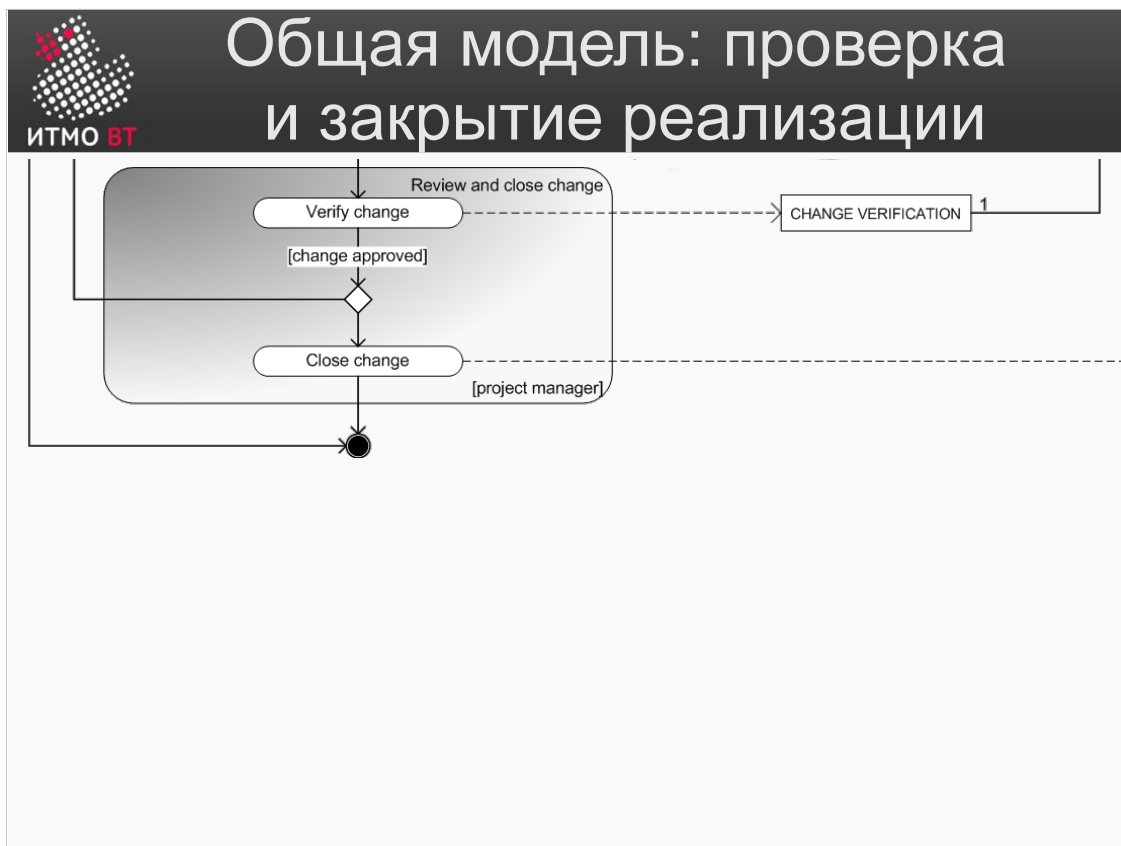
После этого запрос поступает к т.н. комитету по изменениям, который, оценив необходимость изменений и стоимость решения проблемы, меняет статус изменения в Change Request. Если запрос подтверждён, то производится анализ и реализация изменений, в противном случае запрос может быть отменён или отложен.



После принятия решения об изменении анализируется его возможное влияние на другие части системы и возможные последствия для пользователей (Analyze change impact). Необходимо помнить, что в сложных системах внесение изменений в одну часть системы почти всегда приводит к влиянию, и соответственно, изменениям в других частях, поэтому вопрос о влиянии изменения следует тщательно изучить.

После анализа организуется планирование проведения изменений в системе (Create Planning), в ходе которого определяются ресурсы, время и график осуществления действий, необходимых для внесения изменения. В результате этих двух действий появляются артефакты Change Impact Analysis и Change Planning.

Затем изменения передаются на реализацию. Роль, ответственная за неё (Change builder), создает объекты, совокупность которых составляет реализацию изменения (Items), например, исходный код. После этого производится тестирование, вносятся изменения в связанную документацию, и выпускается новый выпуск или версия продукта, «заплатка» и/или конфигурация. При этом создаются артефакты — отчеты о тестировании (Test Report), собственно, документация (Documentation) и выпуск системы (System Release), соответственно.



В конце процесса изменения передаются менеджеру проекта (и иногда — заказчику), проверяются (создаётся артефакт Change Verification), и формально утверждаются.



Системы контроля версий

- Управляют изменениями в программном коде.
- Поддерживают групповую работу.
- Основные типы:
 - На основе файловой системы (с экспортом клиентам).
 - Централизованные (репозиторий на сервере).
 - Распределённые (репозиторий на каждом клиенте с центральным хранилищем на сервере).
- Клиенты встроены в большинство средств разработки.
- SCCS, RCS, ClearCase, CVS, Subversion, MS Visual Source Safe, mercurial, git, Bazaar...

Сами по себе системы контроля версий существуют для управления изменениями в программном коде и поддерживают групповую работу нескольких человек над кодом одновременно, а также контроль над изменениями файлов, создаваемых пользователями системы контроля версий (т.е. программистами).

Существуют три основных типа систем контроля версий:

1) На основе файловой системы. Данный подход является устаревшим. Ранее разработчики пользовались центральным сервером с общим доступом к файлам. Такая система контроля создавала файлы слежения за текущей директорией и позволяла хранить и отслеживать версии только в рамках одной файловой системы. Также было возможно экспортировать файловую систему для доступа удалённых клиентов при помощи сетевых файловых систем (например, NFS).

2) Централизованная, с единым репозиторием — хранилищем исходным кодов проекта на сервере, и удалённым доступом клиентов по специальным протоколам. К таким системам относится Subversion.

3) Распределённая. В ней существует центральный репозиторий, из которого пользователи скачивают данные на свои локальные репозитории. Обратно в центральный репозиторий данные попадают после нескольких стадий локальных проверок. К распределённым системам относится Git.

В первых системах контроля версий (например, SCCS, который был интегрирован во всех потомках Unix System V) все действия производились при помощи командной строки, но в большинстве современных систем контроля версий существуют встроенные клиенты, автоматизирующие работу с версиями и организующие все необходимые действия в набор пунктов меню.

В настоящее время существует большое количество систем контроля версий, таких как ClearCase, CVS, Subversion, MS Visual Source Safe, mercurial, git, Bazaar и т.д.



Одновременная модификация файлов

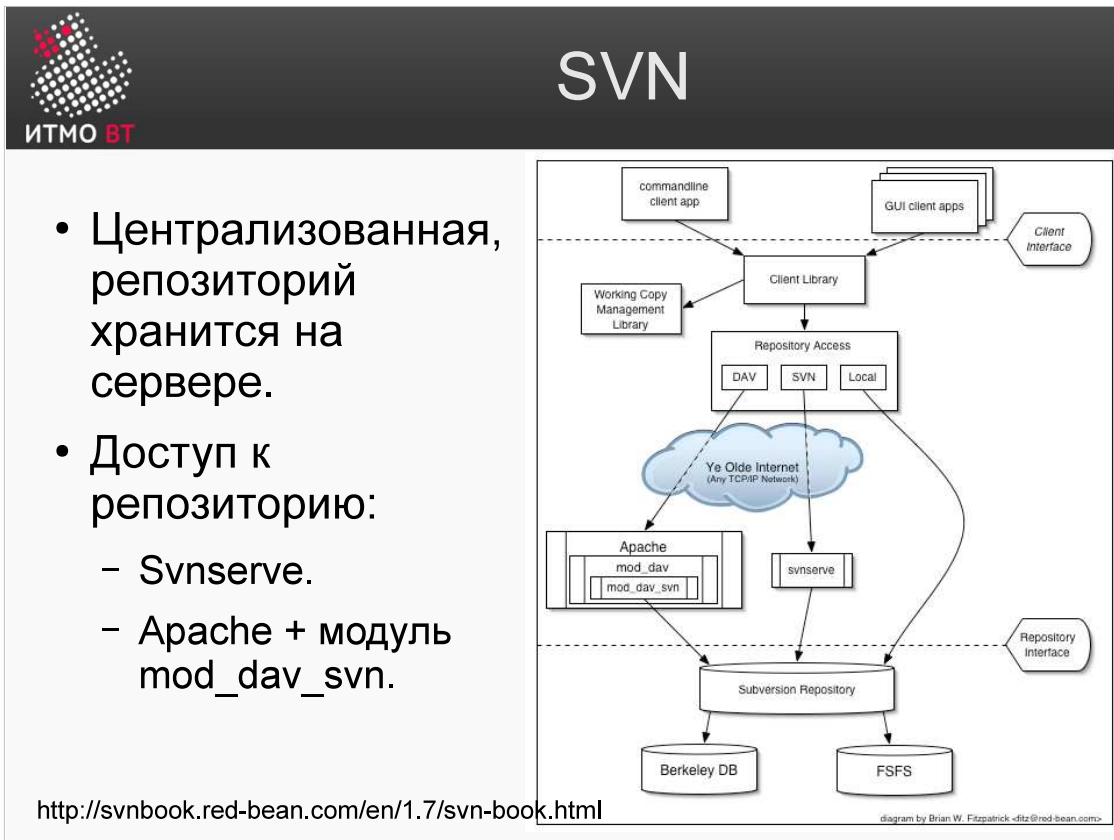
- Lock-modify-unlock:
 - В основном, в СКВ на файловых системах.
 - Замедляет работу команды.
- Copy-modify-merge:
 - Своя рабочая копия у каждого разработчика.
 - Трудности слияния рабочих копий.

Главной проблемой при работе с СКВ является необходимость одновременного редактирования одних и тех же файлов несколькими разработчиками. Одновременное редактирование было технически возможно достаточно давно, но оно приводило к войне правок, когда разработчики в рамках своих заданий вносили взаимно-конфликтующие изменения.

В мире систем контроля версий утвердились два подхода к решению данной проблемы:

1) Подход Lock-modify-unlock, где при работе одного пользователя с файлом, он блокируется для других, и они не могут его модифицировать. Это замедляет работу команды, так как другим пользователям приходится либо ничего не делать, либо переключаться на другие задачи, не связанные с заблокированным файлом. Данный подход был характерен на системах контроля версий, связанных с общей файловой системой.

2) Copy-modify-merge, где каждый пользователь копирует себе весь репозиторий, работает с ним, и изменения всех пользователей сливаются (merge). В данном случае именно слияние рабочих копий разработчиков представляет собой основную проблему. Например, два разработчика могут одновременно изменить один и тот же код двумя различными способами. В этом случае, при попытке сохранить изменения, второй разработчик (тот, который делает коммит по времени вторым, после первого) не сможет этого сделать и будет вынужден разрешать конфликт (во многих случаях — совместно с первым разработчиком).



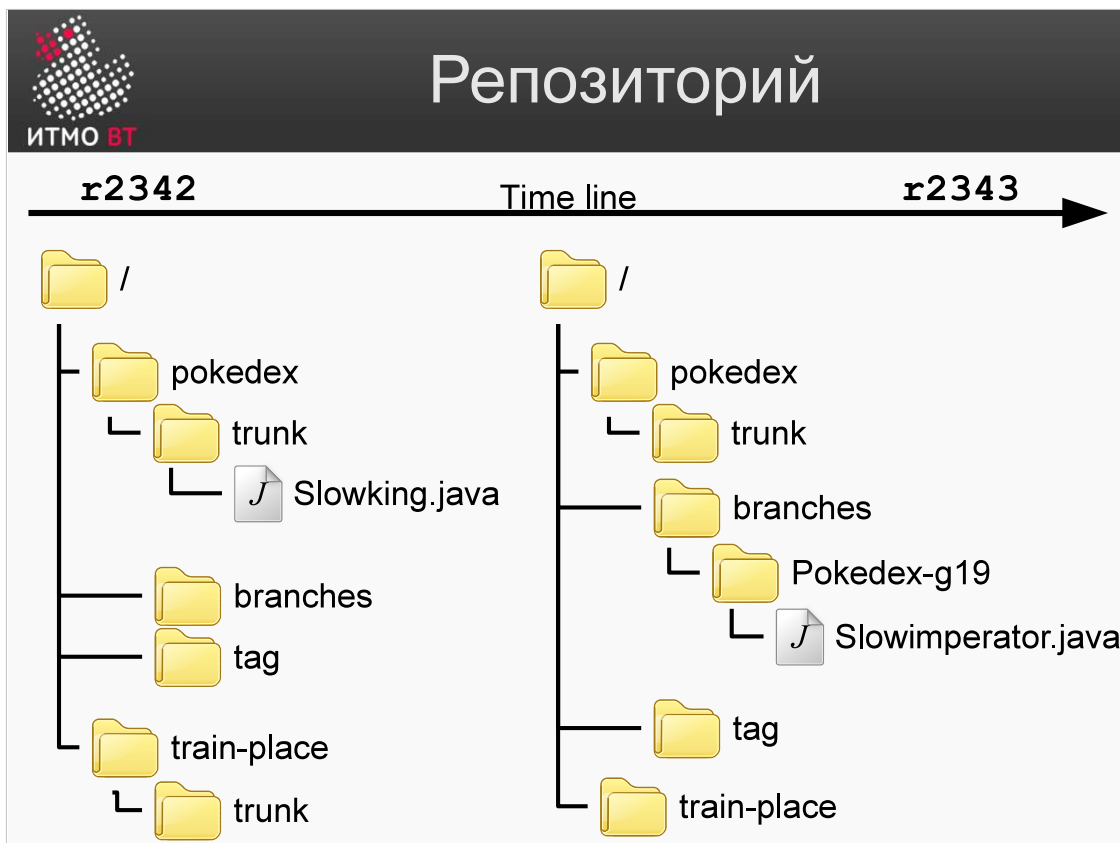
Одной из популярных систем контроля версий является Subversion (SVN).

В архитектуре, которая приведена на слайде, существует уровень хранения репозитория (Subversion repository), который может иметь две технические реализации — размещаться в базе данных Berkeley DB, или, в простейшем случае, в файловой системе (FSFS).

Доступом к репозиторию управляет демон svnserv, который получает команды пользователя и выполняет соответствующие изменения в репозитории. Вторым вариантом доступа является использование сервера Apache и его модулей, реализующих ту же логику, что и svnserv.

Удалённый доступ к репозиторию может осуществляться по нескольким протоколам. Обычно используется протокол svn или связки svn+https и ssh+svn. Этими протоколами может пользоваться клиентская библиотека, которую, в свою очередь, используют приложения, интегрирующие в себя поддержку Subversion.

Клиент SVN не только осуществляет передачу данных с репозитория клиенту и обратно. Он ещё и осуществляет управление локальной копией файлов, которую модифицирует разработчик. Это значит, например, что когда вы скачиваете выбранную версию из репозитория, локальный набор файлов должен быть изменён в соответствии с содержимым выбранной версии.



Репозиторий — это набор файлов проекта(ов), организуемый определённым иерархическим образом для удобной работы с проектом. На слайде показан пример репозитория.

В SVN каждая фиксация изменений (svn commit), пришедшая в репозиторий, повышает версию этого репозитория на 1. Ревизия репозитория — это всегда целое число, уникально характеризующее набор файлов, лежащих в репозитории на какой-то момент времени, задаваемый номером ревизии. В каждый момент времени, при условии успешной фиксации изменений, репозиторий является целостным.

Каждая фиксация изменений svn обязана включать файлы, изменения которых должны быть помещены в репозиторий. Если такое изменение производится из командной строки, то эта строка должна содержать все файлы. В одну фиксацию изменений рекомендуется помещать логически законченное обновление функционала, или, например, исправление одного дефекта. Не рекомендуется объединять разные изменения в один коммит.

Организация файлов обычно такова: существуют каталоги проектов (здесь pokedex и train-place), внутри которых размещены директории, предназначенные для организации работы с разными версиями проекта.

В каталоге trunk происходит основной процесс разработки. В этот каталог разработчики ежедневно вносят результаты своего труда, оформленные в виде целостного набора изменений (с точки зрения и набора файлов, и их логического содержимого). В случае необходимости поставки различных версий разным заказчикам, либо с целью фиксации истории релизов, стабильные на некоторый момент времени копии могут формироваться на основе содержимого каталога trunk и копироваться в дополнительные каталоги branch и tags.



Шпаргалка

- **trunk** — основная ветвь разработки.
- **branches** — хранение модификаций продукта:
 - Releases — значимые версии продукта (3.0,3.5).
 - Features — выполнение работ над версиями со существенными изменениями без влияния на trunk.
 - Vendor — версии с модификациями сторонних библиотек. *Vendor drop*.
- **tag** — функционально целостные изменения:
 - Исправления дефектов ПО (3.5.0-B2947219).
 - Минорные версии (с исправлением нескольких дефектов — 3.5.0.1).

trunk — основная ветвь разработки.

В branches хранятся отдельные модификации, в частности, структурно-значимые версии продукта (например, 3.0 или 3.5). Также в branches можно вести разработку отдельных ветвей реализации, в которых внутренний функционал по сравнению с trunk серьёзно изменён, и которые не следует делать в trunk во избежание серьёзных конфликтов. После проведения всех необходимых изменений, данную функциональную реализацию сливают с основной. Существуют также Vendor branches, версии с модификациями сторонних библиотек, используемых в проекте (и, возможно, изменённых), которые необходимо держать вместе с кодом, который их использует.

В каталогах tags хранятся помеченные версии продукта. С точки зрения SVN каталоги tags используются для работы над исправлением дефектов (3.5.0-B2947219), или, например, для минорных версий с исправлением нескольких дефектов (3.5.0.1).

В SVN для создания новой ветки производится полное копирование файлов старой ветки в новый каталог. При этом точно так же всегда увеличивается версия репозитория.



Основной цикл разработчика

svn checkout — первоначальное создание рабочей копии.

Ночь! Вечер День Утро

- Обновить рабочую копию:
 - **svn update**
- Изменить необходимые файлы:
 - **svn (add, delete, copy, move, mkdir)**
 - Просмотр изменений: **svn status** и **svn diff**
 - Откат изменений: **svn revert**
- Фиксация изменений на сервере:
 - **svn update** для загрузки изменений других.
 - Разрешить конфликты содержимого и структуры файлов.
 - **svn commit** — собственно фиксация.

Основной цикл разработки использует описанные выше свойства SVN.

В начале работы сервер содержит в рабочей ветке все последние накопленные обновления других разработчиков, и в вашем каталоге существует локальная версия предыдущего дня. Чтобы начать ежедневный цикл работы, необходимо скачать с сервера себе в файловую систему обновление локальной копии. При этом скачиваются не только сами данные, но и метаданные. Управляющая информация нужна для обеспечения целостности и для определения того, куда будет производиться коммит. Таким образом, утром рабочего дня каждый разработчик должен выполнить **svn update** и привести свою рабочую копию до того состояния, которое есть на сервере.

Затем производится "великое творение кода" и меняются необходимые файлы. Используются такие команды, как **svn add** (для создания (добавления) файла), **svn delete** (для удаления файла), **svn copy**, и другие (то же самое касается и директорий, а также перемещения файлов внутри рабочей копии).

Если нужно провести откат изменений, используется команда **svn revert**, а команды **svn status** и **svn diff** помогают ориентироваться в изменениях по сравнению с сервером.

Вечером производится фиксация изменений на сервере. Так как другие разработчики тоже могли внести изменения, с сервера снова скачивается версия, там находящаяся. При этом существует вероятность возникновения конфликтов между изменениями данного разработчика и изменениями, сделанными его коллегами. Данные конфликты необходимо разрешить. После этого производится собственно фиксация с помощью команды **svn commit**.

При этом, тому, кто делает первый коммит, не нужно разрешать каких-либо конфликтов, так как он загружает на сервер первое изменение. Также следует отметить, что конфликты между версиями разных разработчиков могут привести и к конфликтам внутри команды. Существует также понятие ночного билда (Nightly build), который собирается именно на основе вечерних коммитов.



Конфликты содержимого файлов

- (e) edit — изменить файл в редакторе.
- (df) diff-full — показать список несовпадений между версиями.
- (r) resolved — подтвердить, что конфликт разрешен в текущей (м.б. отредактированной) версии файла.
- (dc) display-conflict — показать все конфликты.
- (mc) mine-conflict, (tc) theirs-conflict — подтвердить мои (или из репозитория) версии для всех конфликтов.
- (mf) mine-full, (tf) theirs-full — подтвердить версии полностью для файла.
- (p) postpone — отложить «на потом».
- (l) launch — запустить внешнее средство.

```
$ svn update
Updating '.':
U    Pokedex.xml
G    Slowimperator.java
Conflict discovered in 'Slowking.java'
Select:
```

filename.mine — до update
 filename.rOLDREV — до правок
 filename.rNEWREV
 — в репозитории

Основная проблема при фиксации изменений — конфликты содержимого файлов. На слайде приведён пример, в котором при скачивании с сервера возник конфликт между локальной версией разработчика и версией с сервера (например, по-разному поменяли одну и ту же строку).

В случае возникновения конфликта при svn update из всех возможных действий чаще всего выбирается (p) postpone — отложить на потом, так как изменения в этот момент редко возможно редактировать самостоятельно. Следует найти разработчика-автора конфликтных изменений, и согласовать с ним исправления.

При выборе postpone также создаются три файла: filename.mine (файл разработчика до update), filename.rOLDREV (файл из репозитория до правок), и filename.rNEWREV (текущий файл в репозитории). Существуют средства (например triplle diff), при помощи которых можно удобно сравнить одновременно все три файла.



Конфликты содержимого: Diff

Base (1.8)	4/5	Locally Modified (Based On 1.8)
<pre> } private void loadVersioningSystems(Collection<? extends VersioningSystem> systems) { assert versioningSystems.size() == 0; assert localHistory == null; versioningSystems.addAll(systems); for (VersioningSystem system : versioningSystems) { if (localHistory == null && system instanceof LocalHistory) { localHistory = system; } system.addPropertyChangeListener(this); } } private void unloadVersioningSystems() { for (VersioningSystem system : versioningSystems) { system.removePropertyChangeListener(this); } versioningSystems.clear(); localHistory = null; } InterceptionListener getInterceptionListener() { return filesystemInterceptor; } private synchronized void flushFileOwnerCache() { folderOwners.clear(); </pre>	⇒	<pre> loadVersioningSystems(systems); } private void loadVersioningSystems(Collection<? extends VersioningSystem> systems) { versioningSystems.addAll(systems); for (VersioningSystem system : versioningSystems) { if (localHistory == null && system instanceof LocalHistory) { localHistory = system; } system.removePropertyChangeListener(this); } } private synchronized void unloadVersioningSystems() { for (VersioningSystem system : versioningSystems) { system.removePropertyChangeListener(this); } versioningSystems.clear(); // reset all systems localHistory = null; } /** * @return listener for filesystem events */ InterceptionListener getInterceptionListener() { return filesystemInterceptor; } </pre>

<http://wiki.netbeans.org/NewAndNoteWorthyMilestone8>

Обычно в средствах разработки есть собственные средства Diff для сравнения различных версий. Пример приведён на слайде. В нём различия в коде версий выделены разными цветами. Наиболее глубокого анализа требуют не добавленные или удалённые, а изменённые строки.



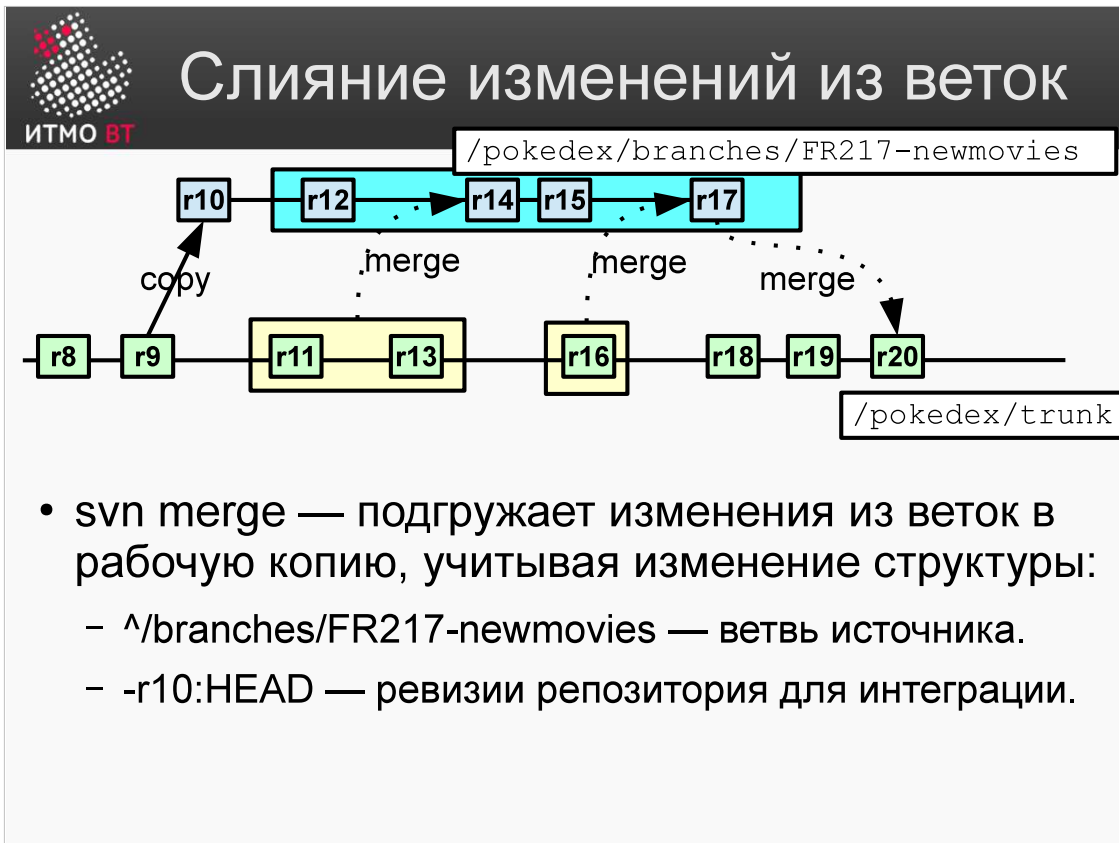
Конфликты структуры

Происходят, когда в репозитории файлы перемещаются, а в рабочей копии они же изменяются.

```
$ svn commit -m "Small fixes"
Sending          pokedex/trunk/Slowimperator.java
svn: E155011: Commit failed (details follow):
svn: E155011: File '/pokedex/trunk/Slowimperator.java' is
out of date
svn: E160013: File not found: transaction '14-e', path
'trunk/Slowimperator.java'
$ svn update
Updating '.':
   C pokedex/trunk/Slowimperator.java
   A pokedex/trunk/SlowImperator.java
Updated to revision 14.
Summary of conflicts:
   Tree conflicts: 1
```

Конфликты такого рода происходят тогда, когда, например, в репозитории перемещаются файлы, а в локальной рабочей копии в эти же файлы вносятся изменения. Иными словами, не совпадает структура программ, и файлы, которые были просто отредактированы на локальной копии, в репозитории переименованы, перемещены, или даже удалены.

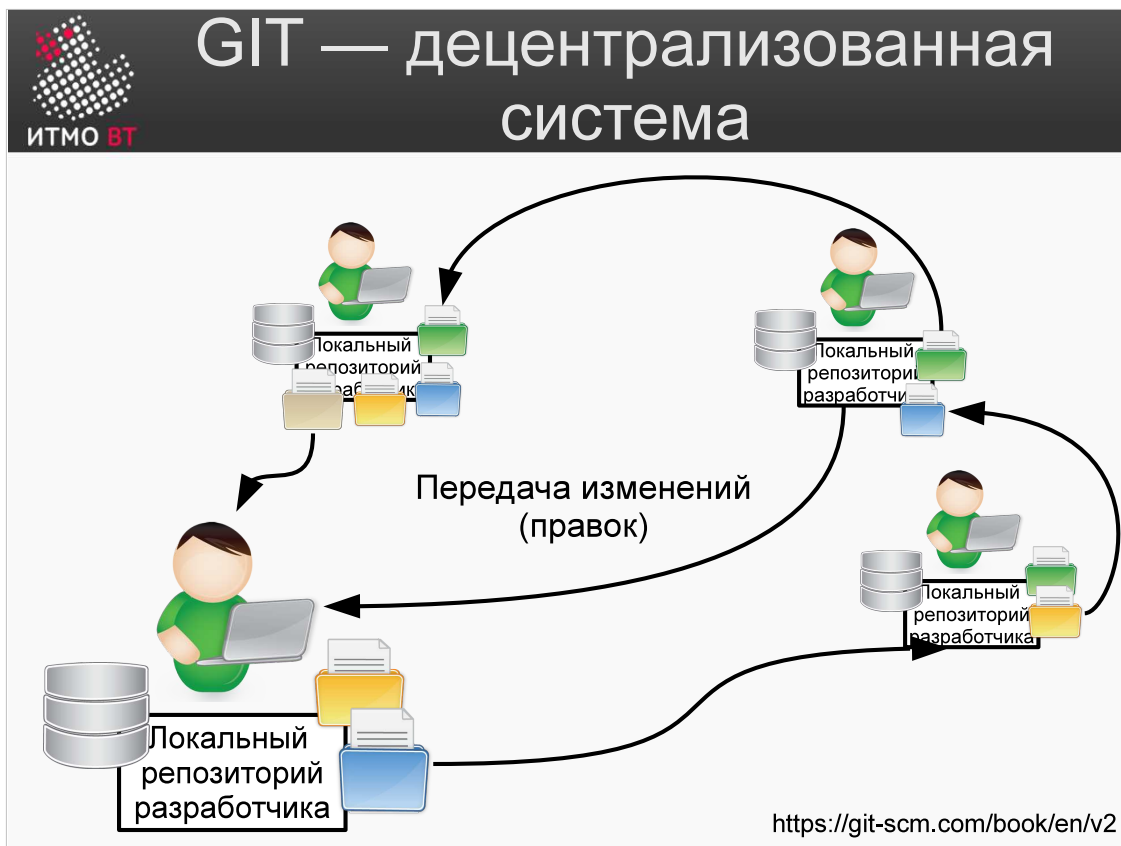
На слайде приведён пример, в котором файл в репозитории был переименован. В локальной копии его имя осталось старым. Конфликт можно разрешить как изменением структуры в локальной копии по образцу версии в репозитории, так и личным общением и согласованием исправлений с тем, кто изменил структуру в репозитории. Не всегда просто определить, в чем заключается конфликт.



При работе над параллельными ветками часто необходимо изменения из одной ветки сливать с изменениями в другой, образуя одну общую ветвь. Для этого предназначена команда `svn merge`.

На примере представлен цикл разработки новых ходов для покемонов. Ревизия r9 была выбрана в качестве исходной, и содержимое ветви /pokedex/trunk было скопировано в ветвь /pokedex/branches/FR217-newmovies. После этого разработка производится в двух, не конфликтующих друг с другом ветвях репозитория (коммиты r11:r13, r15:r16 и r18:r19). В 14 и 17 коммитах из ветви trunk изменения добавляются в ветвь FR217-newmovies. В версии 20 изменения из 17 ветви и изменения коммитов 18 и 19 добавляются в trunk.

При операции слияния возможны конфликты, которые по своему типу не отличаются от описанных на предыдущих слайдах.



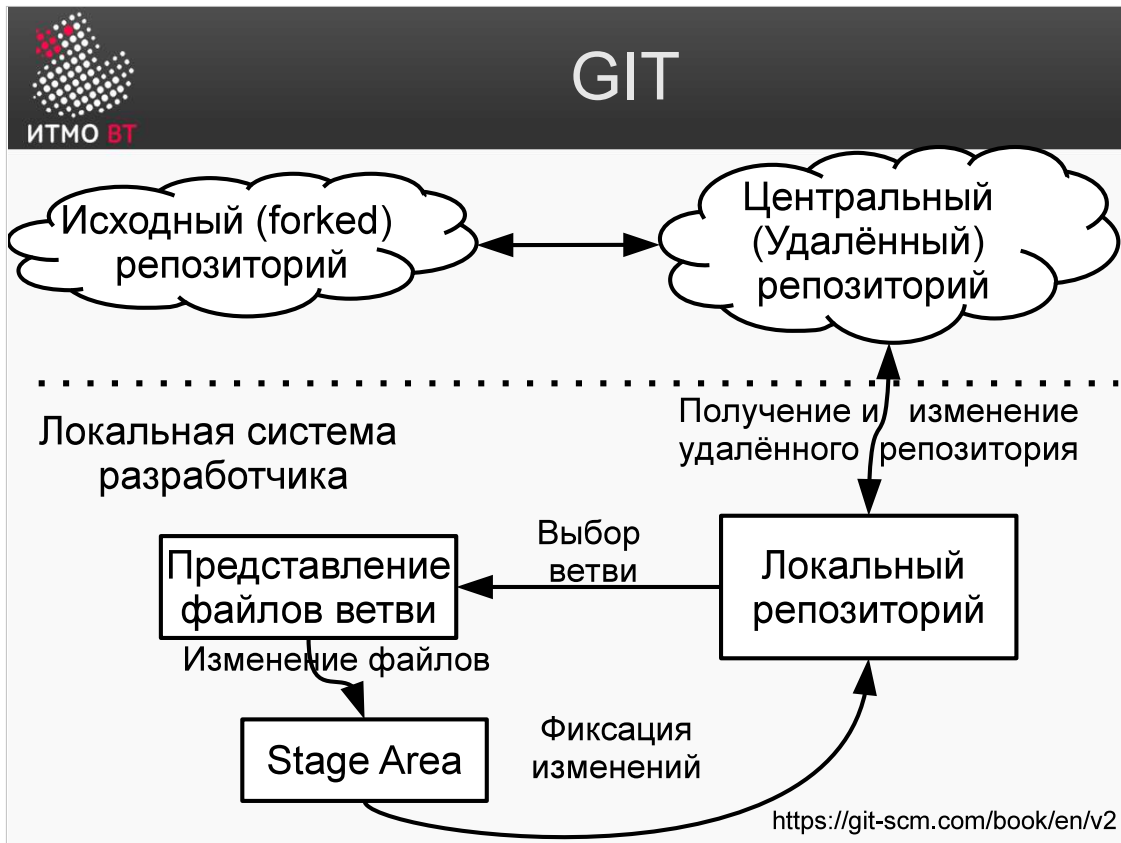
Основной философией одной из самых популярных систем контроля версий — git является ее распределенность. Репозитории существуют локально у каждого разработчика, в них содержится информация о всем проекте целиком. Подчеркнем, что кодовая база, свои изменения и изменения других разработчиков, журналы применения всех доступных изменений находятся у каждого разработчика в локальной копии репозитория проекта.

Когда программист разрабатывает новый функционал (или как говорят на сленге — «фичу»), он может предложить свои правки другим разработчикам, а git предлагает удобный интерфейс по передаче этих правок и внесения их в локальные репозитории коллег-разработчиков. Таким образом может происходить, например, разработка операционных системы — пользователи сами выбирают, какие правки и фичи от других разработчиков им необходимы, чтобы работать над своим функционалом.

Каждый локальный репозиторий имеет свой URL, по которому он может быть доступен, а другие разработчики могут обращаться к нему при помощи алиасов (синонимов), что упрощает команды передачи изменений от разработчика к разработчику и от репозитория к репозиторию.

Благодаря такой архитектуре рабочий процесс совместной работы может быть адаптирован для каждой конкретной команды. В наиболее известном руководстве по Git (ссылка на которое представлена на слайде) определяются три основных рабочих процесса:

- централизованный рабочий процесс — где существует единственный централизованный репозиторий, где все разработчики синхронизируют свою работу с ним;
- рабочий процесс с менеджером по интеграции — где каждый разработчик изменяет канонический репозиторий в своей локальной копии, которую он открывает для других для чтения, а после завершения правок запрашивает интеграционного менеджера загрузить правки из своей копии для чтения в канонический репозиторий;
- рабочий процесс с диктатором и лейтенантами - в основном такой подход используется на огромных проектах, насчитывающих сотни участников; самый известный пример - ядро Linux. Лейтенанты - это интеграционные менеджеры, которые отвечают за отдельные части репозитория. У всех лейтенантов есть свой интеграционный менеджер, который называется доброжелательным диктатором. Репозиторий доброжелательного диктатора выступает как эталонный, откуда все участники процесса должны получать изменения.



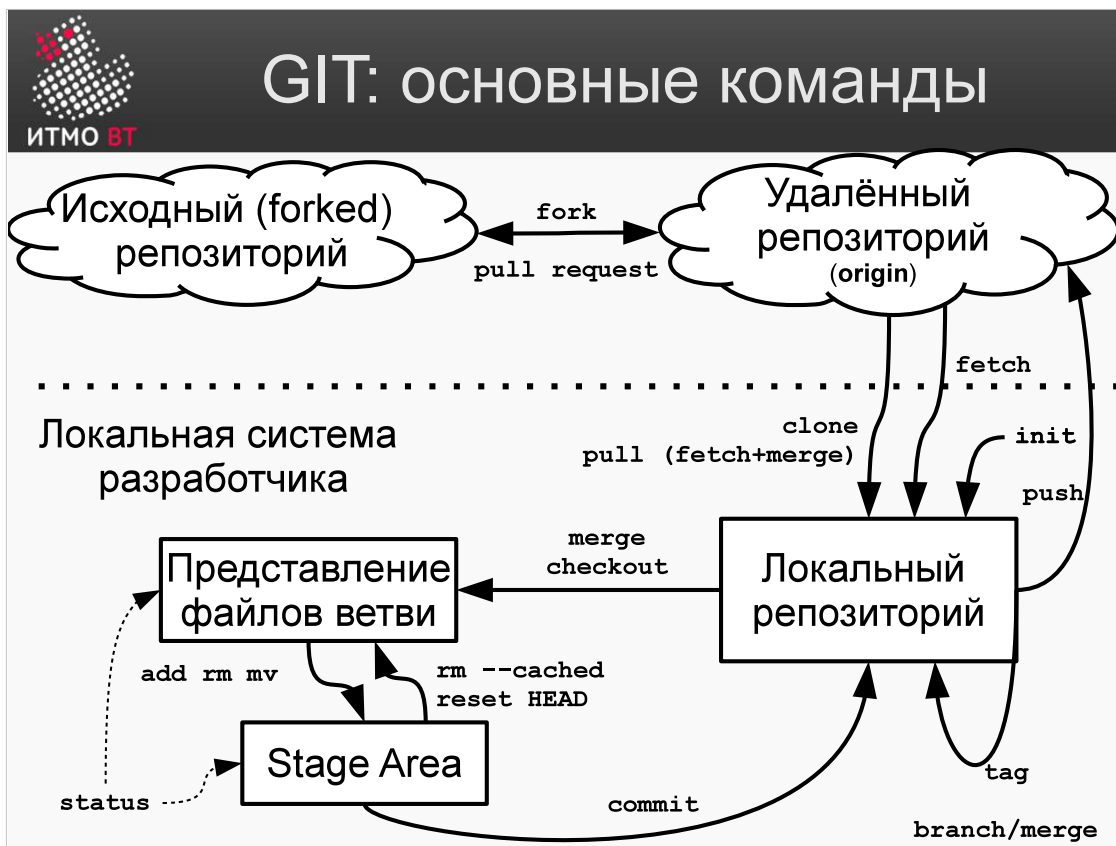
Наиболее распространенным в сообществе разработчиков систем с открытым кодом, использующим популярный ресурс GitHub, является рабочий процесс с интеграционным менеджером. В нем существует центральный удалённый репозиторий, в котором сохраняются результаты работы всей команды разработчиков. Центральный репозиторий может быть получен при помощи организации ветви (fork) от любого другого известного проекта.

Такому центральному репозиторию по умолчанию git назначает алиас origin. Центральный репозиторий для каждого из разработчиков команды является удалённым.

У каждого разработчика есть свой локальный репозиторий, который является клоном (копией) удалённого репозитория. Разработчик выбирает необходимую ветвь в локальном репозитории, получая представление на файловой системе файлов с содержимым ветви, которые он может изменять.

Во время работы с рабочей копией разработчик указывает, какие файлы включить в фиксацию изменений. Эти изменения помещаются в т.н. Stage Area — специальную область, из которой файлы будут включены в коммит. В отличие от svn, здесь нет необходимости указывать все файлы, участвующие в фиксации изменения в одной команде — в git их можно добавить за несколько операций. Затем изменения фиксируются в локальном репозитории. Для помещения информации в центральный репозиторий разработчик использует команду push, при которой, естественно, могут возникать конфликты.

Другой способ фиксации изменений в удалённом репозитории — это конфигурация запросов на фиксацию изменений (pull requests). При этом изменения, которые разработчику необходимо записать, сначала должны быть подтверждены владельцем ветви. Таким образом обычно происходит слияние функциональности в открытом программном коде, управляемым сообществом разработчиков. На слайде это показано двусторонней связью "Удалённого репозитория" и "Исходного репозитория".



На данном слайде приведены основные команды git. Каждая команда создает изменения относительно предыдущего изменения и подсчитывает хеш изменений. Именно поэтому команды git очень быстро выполняются и эффективно обрабатываются системой.

Приведём примеры команд: с помощью команды clone можно скопировать удалённый репозиторий в локальный репозиторий, командой init можно создать локальный репозиторий.

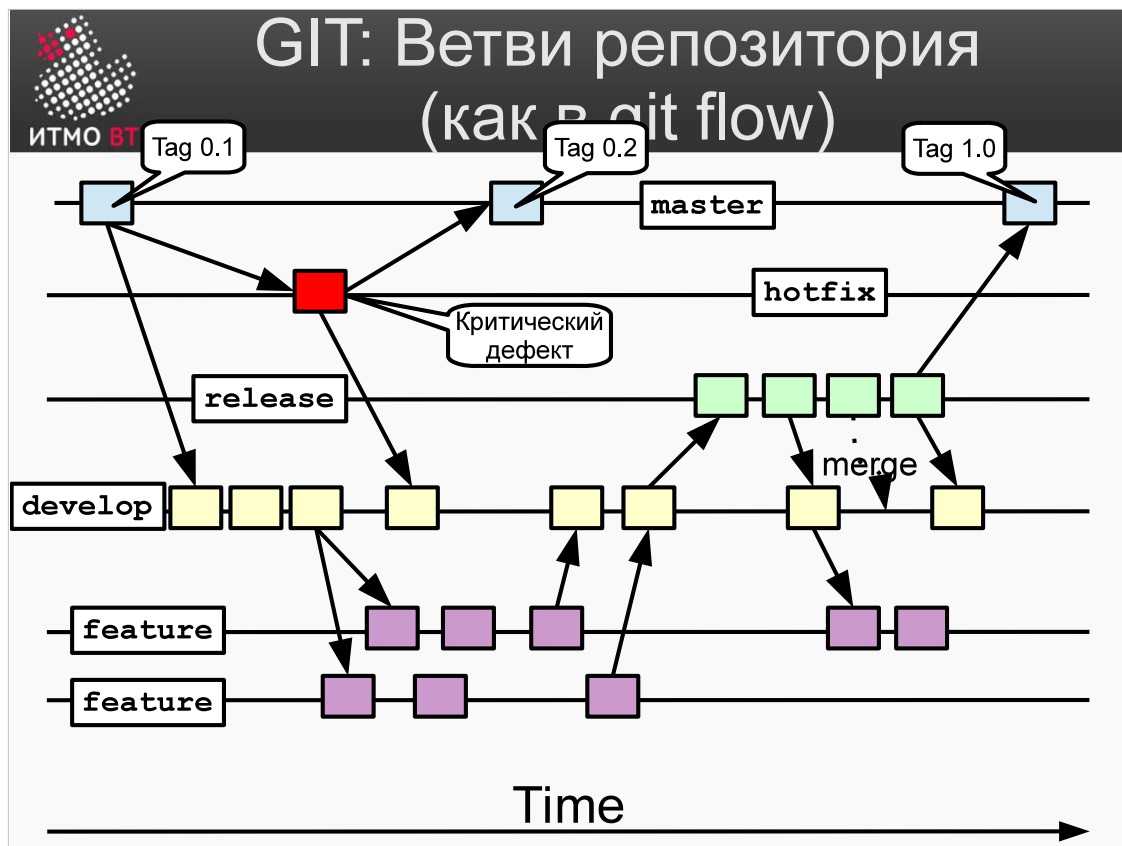
Разработчик может создать новую ветвь при помощи branch, при этом в качестве родительской ветки будет использована ветвь, с которой в настоящий момент происходит работа. Для того, чтобы установить рабочую копию в согласованное с необходимой ветвью состояние (т. е., перейти в эту ветвь), используется команда checkout. Создание ветви не подразумевает переход в нее.

После завершения работы с рабочей копией, разработчик с помощью команды add указывает, какие файлы должны попасть в Stage Area, и с помощью команды commit изменения фиксируются в локальном репозитории. С помощью команды push результаты можно отправить в удалённый репозиторий.

Изменения с удалённого репозитория скачиваются с помощью команды fetch, но рабочая копия при этом автоматически не меняется. Для её обновления нужно выполнить команду merge с желаемой ветвью (использование команды pull равносильно использованию обеих команд вместе). Не рекомендуется использовать команду merge при наличии изменений в рабочей копии.

Другим вариантом включения изменений из удалённого репозитория в рабочую копию может быть использование команд fetch+rebase, в таком случае ваши изменения рабочей копии будут логически применены к другому базовому коммиту.

Команда fork используется для создания ветвления существующего проекта в своем удалённом репозитории. Запрос на применение предлагаемых разработчиком изменений на исходный репозиторий формируется с помощью команды pull request.



Модель разработки в git основна на концепции, что каждое различимое изменение должно разрабатываться в отдельной ветви. Схематично модель ветвлений представлена на слайде.

В ветви Master находятся версии, готовые к поставке пользователю. Эти версии являются основными версиями продукта. Функционал там чётко определен, и список дефектов известен.

Основная разработка происходит в ветви Develop. Данные ветви используют Master как базовую, и в них происходит разработка новой версии продукта. После кодирования все разработчики сливают свои изменения именно в ветвь Develop. При этом изменения рекомендуется делать по функциональным требованиям продукта. Для каждого из требований создается своя ветвь Feature, и там аккумулируются коммиты для этого требования. По мере завершения разработок функционала коммиты из Feature-ветвей подгружаются в ветвь Develop.

Когда накоплено достаточное количество функционала для выпуска новой, готовой к поставке версии, изменения из Develop формируют ветвь Release, а сама ветвь Develop остаётся для дальнейшей разработки новых функций. Ветвь Release используется для исправления тех дефектов, которые препятствуют выходу новой версии. После исправлений готовая версия продукта копируется в ветвь Master. При этом исправления дефектов сливаются в ветвь Develop, так как они там также будут необходимы в дальнейшем.

Ветвь Hotfix предназначена для исправления критических ошибок (т.е. тех, которые препятствуют работе) в версии пользователя. После исправления результат передаётся как в ветвь Master, так и в ветвь Develop.

Процесс повторяется для всех новых функциональных требований.



GIT: Шпаргалка по командам

- `git status`
- `git log [--graph]`
- `git diff`
- `git reset --hard HEAD`
- `git branch`
- `git checkout ветвь`
- `git merge ветвь`
- `git commit -m "сообщение"`
- `git add file`

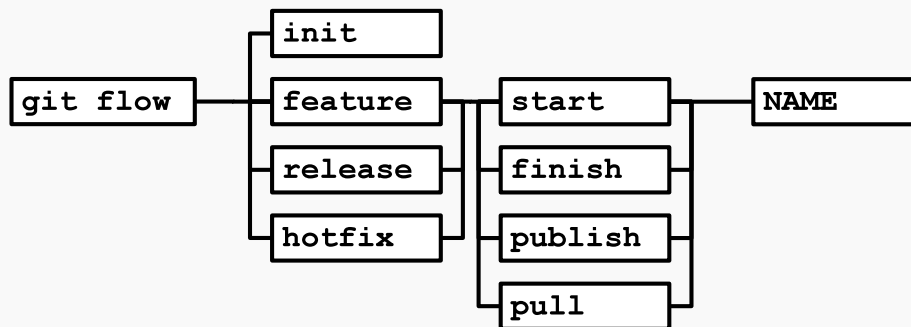
Команды, которые могут понадобиться при выполнении лабораторной работы представлены на слайде. В лабораторной работе функциональность команд сильно урезана, например `git branch` может только показать ветви.

- `git status` — показывает статус текущих состояний файлов в файловой системе и информацию о ветви, в которой производится редактирование.
- `git log` показывает журнал коммитов, а опция `--graph` выводит в графическом виде ветви, в которых производились данные изменение.
- `git diff` — покажет все изменения, которые были сделаны относительно последних зафиксированных изменений.
- `git reset --hard HEAD` сбросит все изменения, которые были сделаны в текущем локальном репозитории.
- `git branch` — показывает ветви.
- `git checkout` — переключает разработчика между ветвями
- `git merge` — объединяет несколько ветвей в текущую ветвь
- `git commit` — фиксирует изменения в текущей ветке. Опция `-m` задает сообщение коммита, которое будет показываться пользователю
- `git add` — добавляет измененные файлы к последующему коммиту, помещая их в Stage Area.



Git flow

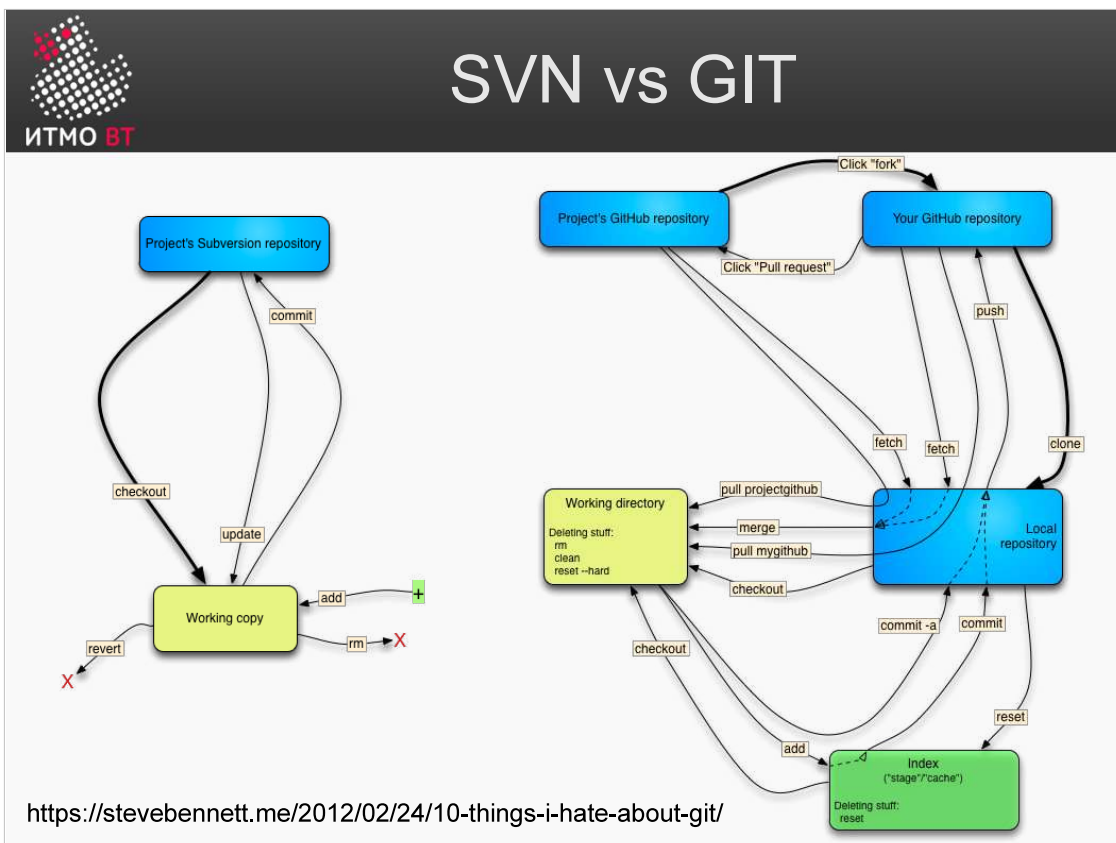
- Плагин для GIT — версионирование в терминах в версий, а не операций.



Каждое проделываемое в Git действие над рабочей копией (создание ветвей, их слияние и т.п.) требует достаточно большого количества команд, и может вызвать у разработчиков сложности с синтаксисом команд. Для того, чтобы упростить эту работу, был создан плагин Git flow, позволяющий работать с версиями именно в терминах версий, а не операций, без точного знания команд и последовательностей действий.

Для использования Git flow необходима определённая начальная настройка репозитория. Для этого у Git flow есть функция `init`, которая при старте задаёт пользователю несколько вопросов и производит необходимую настройку.

Для создания новой `feature` в git flow используется команда `git flow feature start FEATURE_NAME`. Когда надо указать на окончание изменений, используется `git flow feature finish FEATURE_NAME`. Остальные команды формируются аналогичным образом. Все это позволяет пользоваться репозиторием на более высоком, независимом от синтаксиса git уровне, при этом реализуется модель, указанная на предыдущих слайдах.



Для завершения обсуждения сравнительных характеристик SVN и GIT на слайде представлены два графа работы с обеими системами контроля версий. Как видно из диаграмм, GIT использует гораздо больше команд, чем SVN, и требует в повседневной работе большего количества действий и знаний от разработчика.